

Практическая работа №7

Выполнила: Ушакова Е.А. гр. ЗП-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке
<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Новый класс ChartBuilder. Заголовочный файл:

```
1  #ifndef CHARTBUILDER_H
2  #define CHARTBUILDER_H
3
4  #include <QGraphicsScene>
5  #include <QTableWidget>
6
7  class ChartBuilder : public QGraphicsScene
8  {
9  public:
10     explicit ChartBuilder(QTableWidget *sourceTable, QGraphicsView *parent = nullptr);
11     void contextMenuEvent(QGraphicsSceneContextMenuEvent *event);
12     void buildBarChart(const QModelIndexList &selectedCells);
13     void exportToPng(const QString &path);
14
15 private:
16     QTableWidget *m_sourceTable;
17     QMap<QString, double> m_data;
18
19 };
20
21 #endif // CHARTBUILDER_H
```

Вызов функций происходит из контекстного меню TableDocument, там же новый объект добавляется на экран в виде отдельной вкладки m_tabs

```
    emit contentChanged(true);
} else if (selectedAction == actionCreateBarChart && !table->selectedItems().isEmpty()) {
    auto *chart = new ChartBuilder(table);
    chart->buildBarChart(table->selectionModel()->selectedIndexes());
    QGraphicsView *chartView = new QGraphicsView(chart);
    int tabIndex = m_tabs->addTab(chartView, tr("Diagramm"));
    m_tabs->setCurrentIndex(tabIndex);
    emit contentChanged(true);
}
}
```

2) в функцию buildBarCharts передается только выделенная часть таблицы.

Перед началом отрисовки задаются изначальные размеры таблицы, а также значения ширины столбца и отступов между столбцами.

```
26
27 void ChartBuilder::buildBarChart(const QModelIndexList &selectedCells) {
28     clear();
29     m_data.clear();
30     for (int i = 0; i < selectedCells.size(); i = i + 2) {
31         QString key = m_sourceTable->item(selectedCells[i].row(),selectedCells[i].column())->text();
32         double value = m_sourceTable->item(selectedCells[i + 1].row(),selectedCells[i + 1].column())->text().toDouble();
33         m_data.insert(key, value);
34     }
35     int chartMaxHeight = 400;
36     int barWidth = 100;
37     int spacer = 10;
38     double chartWidth = barWidth * m_data.size() + spacer * (m_data.size() + 1);
39     double maxValue = 0.0;
40
41     for (auto i = m_data.cbegin(),end = m_data.cend(); i != end; ++i) {
42         qDebug() << qPrintable(i.key()) << ": " << i.value() << "\n";
43         if (i.value() > maxValue) {
44             maxValue = i.value();
45         }
46     }
47
48     qDebug() << "max: " << maxValue;
49
50     QGraphicsRectItem *bar;
51     QGraphicsLineItem *baseline = new QGraphicsLineItem(-chartWidth / 2, chartMaxHeight, chartWidth / 2, chartMaxHeight);
52     addItem(baseline);
53     QGraphicsTextItem *caption;
54     double posX = -(chartWidth / 2) + spacer;
55
56     for (auto i = m_data.cbegin(),end = m_data.cend(); i != end; ++i) {
57         qDebug() << "cur X:" <<posX;
58         double height = ((chartMaxHeight - 20) / maxValue) * i.value();
59         bar = new QGraphicsRectItem(posX, chartMaxHeight - height, barWidth, height);
60         caption = new QGraphicsTextItem(i.key() + QString("\n(%1)").arg(i.value()), bar);
61         //qDebug() << "capt height: " << caption->boundingRect().height();
62         caption->setPos(posX, chartMaxHeight + spacer);
63         caption->setTextWidth(barWidth);
64         //qDebug() << "capt new height: " << caption->boundingRect().height();
65         // QGraphicsItemGroup *group = createItemGroup({bar, caption});
66         // addItem(group);
67         addItem(bar);
68         addItem(caption);
69         posX = posX + barWidth + spacer;
70     }
71
72 }
73 }
```

Все элементы перебираются попарно и загружаются в QМар, где первый элемент пары это ключ а второй значение.

Среди полученных пар находится максимальное значение, от которого будут масштабироваться элементы диаграммы.

Далее с помощью QGraphicsLineItem рисуется базовая плоскость диаграммы.

Изначальная позиция по X вычисляется до входа в цикл.

Затем происходит заполнение диаграммы. Для каждой пары создается столбец и подпись, которые добавляются на сцену. Далее позиция инкрементируется для следующего элемента.

```
18 void ChartBuilder::contextMenuEvent(QGraphicsSceneContextMenuEvent *event) {
19     QMenu menu;
20     QAction *actionExportPNG = menu.addAction(tr("Export to PNG"));
21     QAction *actionCloseDiagram = menu.addAction(tr("Close Diagram"));
22     QAction *selectedAction = menu.exec(event->screenPos());
23     if (!selectedAction) return;
24     if(selectedAction == actionExportPNG) {
25         QString path = QFileDialog::getSaveFileName(nullptr, tr("Save Diagram as PNG"), QString(), tr("PNG Files (*.png)"));
26
27         if (!path.isEmpty()) {
28             exportToPng(path);
29         }
30     } else if (selectedAction == actionCloseDiagram) {
31         closeDiagram();
32     }
33 }
34 }
```

3) Экспорт изображения. Переопределен метод contextMenuEvent. При выборе действия сохранить в PNG запрашивается путь сохранения, после чего экспортируется сцена

В методе exportToPng проверяется, что список не пуст, затем рисуется изображение по размеру сцены и заливается белым фоном. Затем с помощью painter отрисовывается сцена и

```

87
88 void ChartBuilder::closeDiagram() {
89     clear();
90     m_items.clear();
91     m_data.clear();
92 }
93 qDebug() << "Diagram closed";
94
95
96 void ChartBuilder::exportToPng(const QString &path) {
97     if (m_items.isEmpty()) {
98         QMessageBox::warning(nullptr, tr("Export Error"), tr("No diagram to export"));
99         return;
100     }
101     QRectF sceneRect = this->itemsBoundingRect();
102
103     qreal margin = 50;
104     sceneRect.adjust(-margin, -margin, margin, margin);
105
106     QImage image(sceneRect.size().toSize(), QImage::Format_ARGB32);
107     image.fill(Qt::white);
108
109     QPainter painter(&image);
110     painter.setRenderHint(QPainter::Antialiasing);
111     this->render(&painter, QRectF(), sceneRect);
112     painter.end();
113
114     if (image.save(path, "PNG")) {
115         QMessageBox::information(nullptr, tr("Export Success"),
116                                 tr("Diagram exported successfully to:\n%1").arg(path));
117         qDebug() << "Diagram exported to:" << path;
118     } else {
119         QMessageBox::warning(nullptr, tr("Export Error"),
120                             tr("Failed to save diagram to:\n%1").arg(path));
121         qDebug() << "Failed to export diagram to:" << path;
122     }
123 }
124
125
126

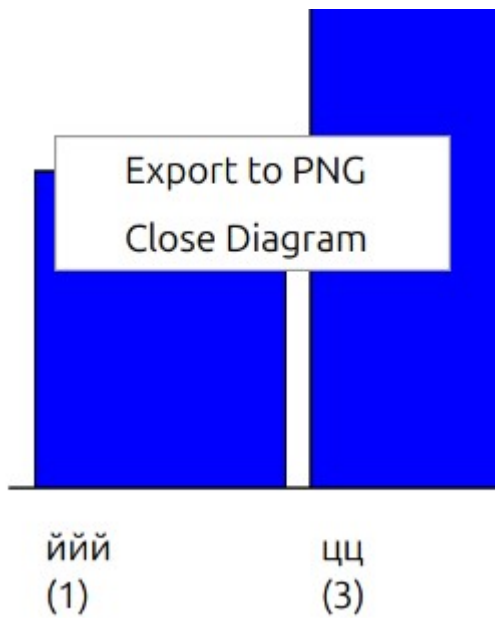
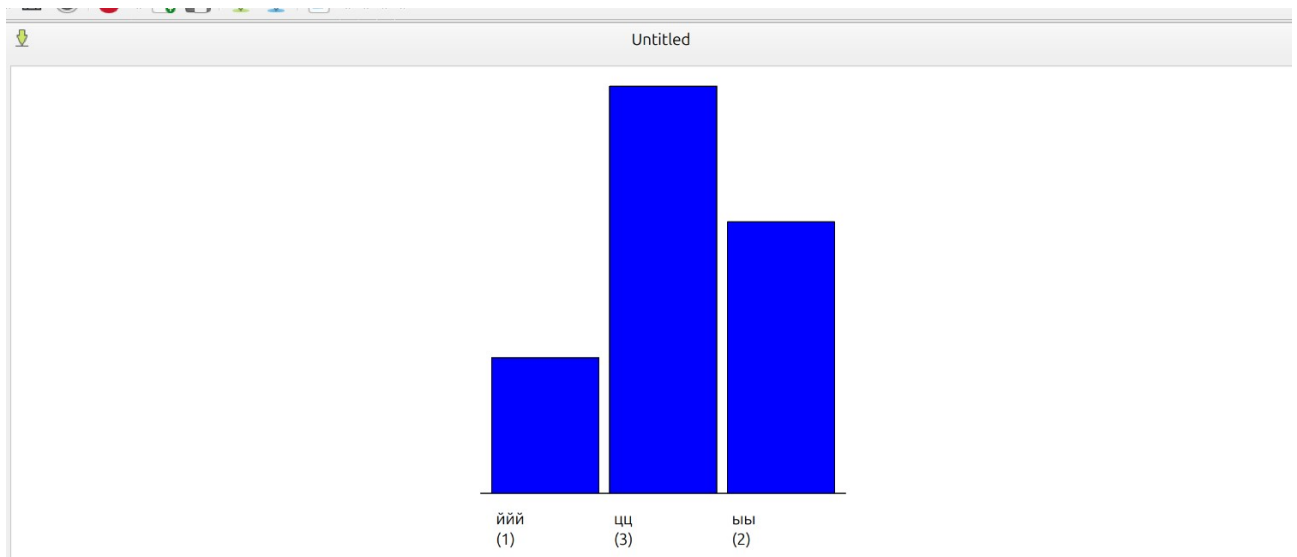
```

изображение сохраняется по указанному пути.

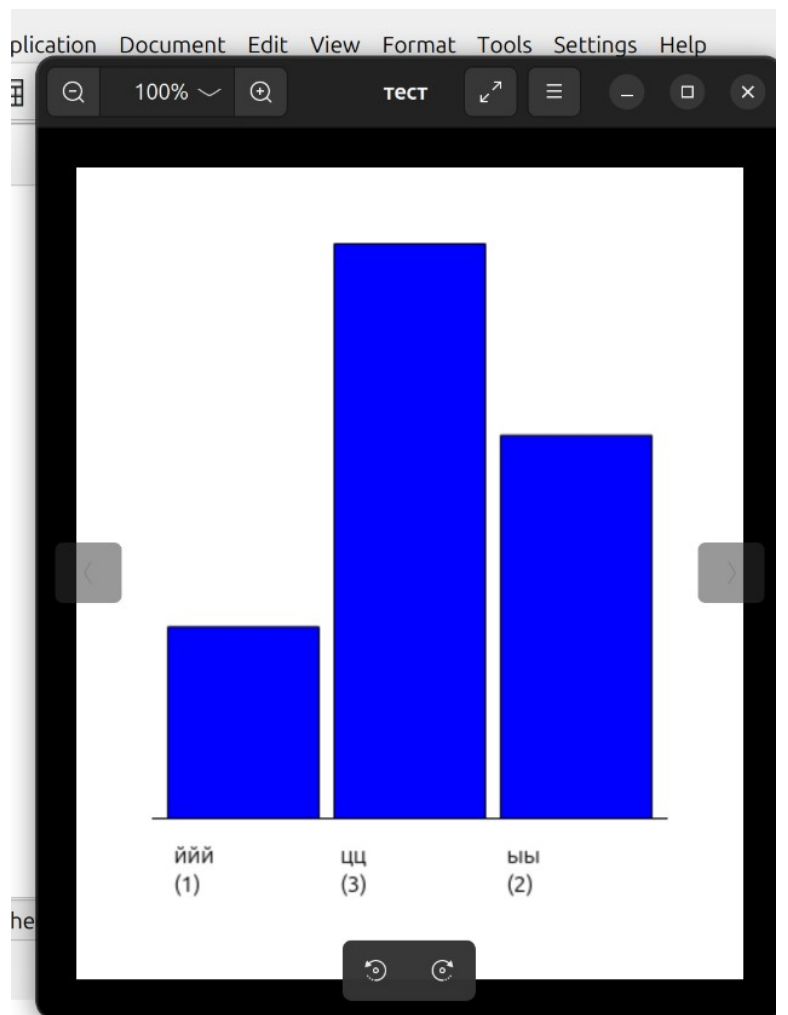
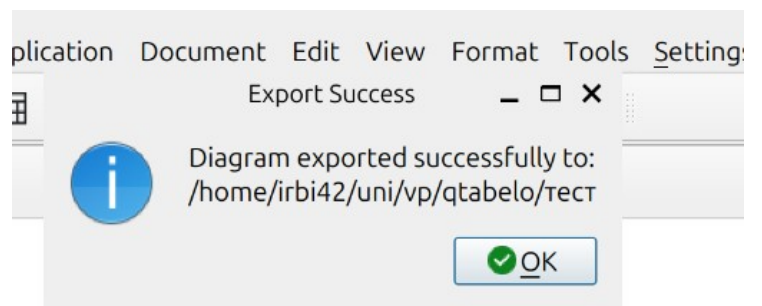
Работа программы:

	Column 1	Column 2	Column 3	Column 4	Column 5
1	ййй	1			
2	ыы	2			
3	цц	3			
4					
5					
6					
7					
8					

Add Row
Remove Row
Add Column
Remove Column
Create Bar Chart



Export to PNG
Close Diagram



properties_pages.cpp cur X: -160
recent_document_list.cpp QGraphicsScene::addItem: item has already been added
rename_dialog.cpp cur X: -50
 QGraphicsScene::addItem: item has already been added